

LI2: A New Learning-Based Approach to Timely Monitoring of Points-of-Interest With UAV

Ziyao Huang , Weiwei Wu , *Member, IEEE*, Kui Wu , *Senior Member, IEEE*, Hang Yuan ,
Chenchen Fu , *Member, IEEE*, Feng Shan , Jianping Wang , *Fellow, IEEE*, and Junzhou Luo , *Member, IEEE*

Abstract—Unmanned aerial vehicles (UAVs) play a critical role in disaster response, swiftly gathering information from various points-of-interest (PoIs) across extensive areas. The freshness of this information is measured by the age of information (AoI), representing the time since the latest information acquisition of a specific PoI. However, devising AoI-minimizing routes for UAVs in obstructed post-disaster environments poses unique challenges that have yet to be fully overcome. Obstacles, like post-disaster barriers, can impede direct flight paths between PoIs, and limited battery life requires energy-conscious route planning. Additionally, existing solutions fail to universally minimize varying data freshness requirements. This research addresses the AoI-driven UAV travel problem, seeking to establish periodic routes that optimize AoI metrics while considering energy and general graph constraints. We develop a learning-based algorithm to enhance the current route iteratively, utilizing guidance from a deep reinforcement learning (DRL) agent and executing a series of operations to potentially decrease AoI while adhering to topological and energy constraints. The algorithm is validated on real post-disaster datasets, demonstrating significant improvements in various AoI metrics compared to other learning-based approaches. Furthermore, our algorithm outperforms approximation algorithms and can approach the global optimum when tailored to existing AoI-minimizing problems.

Index Terms—AoI, path planning, UAV, reinforcement learning, disaster response.

I. INTRODUCTION

DISASTER response is critical for mitigating the repercussions of natural and human-induced calamities. The

Received 15 December 2023; revised 1 July 2024; accepted 3 September 2024. Date of publication 17 September 2024; date of current version 4 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62232004, under Grant 61972086, under Grant 62272099, under Grant 62202100, under Grant 62072101, and under Grant 62132009, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20230024 and under Grant BK20231543, in part by the Hong Kong Research Grant Council under Grant GRF 11218621, and in part by the Key Laboratory of Computer Network and Information Integration (Southeast University), Ministry of Education. Recommended for acceptance by C. Xin. (Corresponding author: Weiwei Wu.)

Ziyao Huang is with the Department of Computer Science and Engineering, Southeast University, Nanjing 211102, China, and also with the Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong (e-mail: seuhuangziyao@outlook.com).

Weiwei Wu, Hang Yuan, Chenchen Fu, Feng Shan, and Junzhou Luo are with the Department of Computer Science and Engineering, Southeast University, Nanjing 211102, China (e-mail: weiweiwu@seu.edu.cn; 220222154@seu.edu.cn; chenchenfu@seu.edu.cn; shanfeng@seu.edu.cn; jluo@seu.edu.cn).

Kui Wu is with the Department of Computer Science, University of Victoria, Victoria, BC V8P 5C2, Canada (e-mail: wkui@uvic.ca).

Jianping Wang is with the Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong (e-mail: jianwang@cityu.edu.hk).

Digital Object Identifier 10.1109/TMC.2024.3461708

cornerstone of effective disaster management is the swift and accurate collection of information, which enables decision-makers to allocate resources judiciously and direct rescue operations as needed. Recently, UAVs have come to the forefront as crucial resources in disaster response due to their inherent mobility and ability to penetrate difficult-to-reach regions. For example, following an earthquake or a fire, a UAV, outfitted with state-of-the-art sensors, can hasten the search and rescue of survivors stranded in unreachable zones. In addition, UAVs can gauge the damage inflicted on vital infrastructure such as bridges and dams, thereby aiding informed evacuation strategies. Within these contexts, UAVs follow predetermined routes to repeatedly inspect PoIs, consistently providing updated information that augments the situational awareness of emergency responders and enables them to make well-informed decisions.

The freshness of information is typically gauged using the AoI metric, defined as the time since the most recent successful information acquisition from the monitored target. To ensure persistent and timely surveillance of PoIs during disaster response, the UAV's path must be carefully designed to optimize AoI performance. Numerous studies have focused on AoI-centric data collection and UAV scheduling. However, none of these studies adequately address the unique requirements within the context of UAV-assisted disaster response due to three primary reasons:

- First, arbitrary terrains necessitate a general graph network. Several recent studies have delved into AoI-centric route optimization problems for mobile agents responsible for data collection from a set of nodes [1], [2], [3]. However, most of these assume an obstacle-free plane, indicating that any two PoIs can be reached directly within their Euclidean distance. This assumption doesn't apply in post-disaster scenarios, as the environment is often hazardous and cluttered. The mobility constraint has been modeled by a graph in some research [4], but these studies typically consider a unit distance graph, neglecting the limited energy capacity of UAVs.
- Second, disaster response often imposes varying requirements on the “freshness” of the overall information. In some instances, the focus may be on the time-average AoI. Conversely, when certain PoIs are more critical than others, the emphasis may be on the maximum weighted AoI. Hence, a unified solution that caters to different practical AoI metrics is required.
- Lastly, in terms of UAV scheduling, most work either presumes obstacle-free conditions (e.g., [5], [6], [7]) or

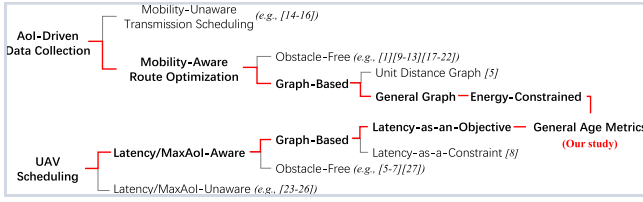


Fig. 1. Position of our work in the literature.

considers latency as a constraint (e.g., [8]) rather than an optimization objective.

Therefore, there is a pressing need to address a unique problem that has remained untouched in existing literature, as depicted in Fig. 1. More specifically, the route optimization problem entails collecting information about heterogeneous PoIs located on a plane with different weights to optimize a given age metric, while also considering energy and mobility graph constraints. The energy constraint necessitates the UAV returning to the Ground Control Station (GCS) for recharging before its energy depletes. The general mobility graph outlines feasible routes for monitoring and recharging by identifying viable edges among PoIs and the GCS, along with the travel time and energy cost on each edge.

Technical Challenges: Despite a wealth of research conducted on AoI in the past decade, the existing methodologies are not adequately equipped to address the above problem. Predominantly, existing studies concentrate on transmission scheduling problems, wherein links to the information holder are activated under interference constraints. These constraints limit the links that can be concurrently activated. However, our problem brings in a new dimension - the spatial distribution of PoIs constrains the links that can be successively activated.

Methodologically, the prevailing strategy for tackling AoI-related issues with spatially distributed PoIs is DRL [1], [9], [10], [11], [12], [13], thanks to the power of DRL in solving optimal control problems involving massive action space. In the majority of these studies, the DRL algorithm learns the next position to go [9], [10], [12], [13] or the next PoI to visit directly [1], [11]. Nevertheless, due to the lack of strict constraint handling, DRL is less effective in satisfying hard constraints critical for disaster response applications. As a result, these existing solutions face difficulties in satisfying the crucial graph/energy constraints in our problem.

Contributions: This paper tackles the above challenges with a novel learning-based insert-then-improve (LI2) approach, which establishes a periodic route for the UAV and retains the robust exploration ability of DRL in complex environments while adhering to the graph and energy constraints. To summarize the idea, the algorithm initiates from an initial route, then iteratively: 1) inserts a PoI (whose index is provided by the DRL algorithm) into the position that produces the optimal AoI performance among all feasible insertion positions within the current route, and 2) attempts to enhance the current route by repeatedly invoking predefined improvement operators. These operators enumerate potential manipulations, for example, relocating one PoI to a new position, for the current route and preserving the best

feasible route. This proposed framework also includes a new idea sharply different from existing DRL-embedded optimization algorithms for classical routing problems, as further elaborated in Section II-C. Overall, we make the following contributions:

- To the authors' best knowledge, this is the first work in the literature that addresses the AoI optimization problem on a general mobility graph with the energy constraint and adapts to various AoI metrics.
- We propose a general algorithm framework that uses DRL to cope with the AoI-driven route optimization problem subject to constraints. The framework cycles through a loop of Insertion, Improvement, and Enforcement stages. This methodology is versatile enough to address a broad range of routing problems, where additional problem-specific expert knowledge can be incorporated by integrating a more customized improvement operator.
- Our solution has undergone rigorous testing using real-world post-disaster datasets, consistently demonstrating its superior AoI performance compared to existing DRL-based baselines. In application to UAV traveling problems specifically aimed at minimizing AoI, our solution has not only outperformed approximation algorithms but has also come remarkably close to achieving the AoI lower bound. The efficiency of our solution has been further validated via field testing in a real environment.

II. RELATED WORK

As illustrated in Fig. 1, our work is related to AoI and UAV scheduling. Regarding the solution methodology, our work is relevant to DRL-based routing algorithms.

A. The Work on AoI

Most studies have focused on optimizing the Age of Information (AoI) in system transmission scheduling problems [14], [15] or meeting a given AoI constraint [16]. However, they often limit transmission concurrency due to channel interference, which makes them unable to handle the spatial distribution of Points of Interest (PoIs) that requires consecutive visits.

Recent research addresses the spatial distribution of PoIs in AoI-driven data collection [1], [9], [10], [11], [12], [13], [17], [18], [19], using DRL alongside factors like energy and throughput. But these studies often use DRL to determine the next position or PoI to visit [1], [9], [10], [11], [12], [13], [17], [18], [19], which doesn't address the non-complete reachability between PoIs (i.e., the graph constraint), important in modeling cluttered disaster response environments. Some other studies have developed heuristic algorithms for AoI-driven data collection [20], [21], [22], but these also overlook the graph constraints.

One study managed the graph constraint [4], but they assumed all edges were the same length. Also, they didn't consider recharging operations or short-term AoI performance, thereby ignoring the energy constraints of UAVs.

B. The Work on UAV Scheduling

There's a lot of research on UAV scheduling problems, which involve planning the routes of one or more UAVs to complete tasks on a graph. For example, [23] proposed an ant colony-based algorithm to improve the flight path of UAVs in order to optimize the weighted summation of monitored vessels. [24] investigated the problem of scheduling a set of drones on a graph to maximize the number of demands at nodes that are met by drone visits. [25] designed a UAV scheduling algorithm to maximize the collected profit at visited nodes in the context of drone delivery. [26] studied the drone scheduling problem to maximize the environmental benefits by conducting periodic tasks. These studies focus on metrics related to visiting frequency of node or edge. Conversely, the AoI of a node depends on not only the visiting frequency of nodes or edges but also the inter-visit time. The failure to consider this additional factor prevents these studies from effectively optimizing the AoI of nodes.

The most related sub-problem within UAV scheduling is persistent monitoring, which aims to minimize the maximum visiting latency (the longest time between visits) of any node. This problem was first explored in robot path planning [5] and expanded upon in subsequent research [6], [7], [27]. Yet, all of these studies assumed a free-of-obstacles environment. Only one study considered energy and graph constraints [8], but it aimed to find a feasible route, not to optimize an objective. Also, these studies mainly address the maximum weighted AoI metric, which may not be applicable to other AoI metrics like time-average AoI.

C. DRL-Based Routing Algorithms

Due to the complexity and scalability issues of existing solutions, people have started using Deep Reinforcement Learning (DRL) to tackle vehicle routing problems, including variants such as the Traveling Salesman Problem (TSP). Most DRL-based TSP routing algorithms either employ DRL as an end-to-end solution to directly output the route [28], [29], [30], [31], [32], or combine the DRL technique with existing heuristics [33], [34], [35], [36], [37], [38]. For example, [28] demonstrated that combining an attention encoder with a long short-term memory (LSTM) decoder could improve both solution quality and computational efficiency, and [31] treated the TSP as a computer vision problem and proposed generating the route via a deep convolutional network. Studies such as [33] and [34] integrated DRL modules with the state-of-the-art heuristic algorithm for TSP, LKH3, to enhance its performance. Furthermore, [35] and [36] combined the DRL technique with genetic algorithms to iteratively find better solutions. These methods primarily focus on the visiting order, ensuring each node is visited exactly once. However, the AoI of a node is significantly influenced by the inter-visit time of nodes, suggesting that nodes with higher priority may need to be visited multiple times. Furthermore, most studies do not address the graph and energy constraints encountered in UAV-based post-disaster response. Thus, our problem is significantly more complex and cannot be solved with existing DRL-based routing algorithms.

The closest DRL-based routing algorithm to our issue is [39]. This algorithm starts with a random feasible route and iteratively improves it, sometimes adding random perturbations to avoid getting stuck in local optima. We also use predefined operators to enhance our route, but our approach differs in two aspects: (1) we start from a reasonably good solution to speed up convergence, and (2) we use DRL to learn the index of the Point of Interest (PoI) to insert, adapting to a variable node count.

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. Weighted Graph Model

We consider a typical aerial monitoring scenario in the aftermath of a disaster where a UAV moves to collect data from PoIs on the ground and simultaneously streams the data back to its GCS to monitor the real-time status of PoIs.

Formally, the nodes on the ground, including the GCS and PoIs, are denoted by the set $\mathcal{K} = \{0, 1, 2, \dots, K\}$, where 0 is the index of the GCS, and other K indices denote the PoIs. In disaster response, the priorities of PoIs may vary depending on the situation. For instance, in the aftermath of an earthquake, the emergency response system may consider factors such as crowd density and the extent of damage to prioritize buildings and facilities, enabling the efficient formulation of rescue and evacuation plans. Similarly, prioritizing areas based on the pattern and density of forest and land fires is crucial for a targeted and effective fire risk monitoring system. Therefore, each PoI $k \in \mathcal{K} \setminus \{0\}$ is assigned a positive weight $w_k \in (0, 1]$ to reflect its priority. Note that we can set w_0 to zero if we do not need to collect information from GCS. Otherwise, we assume GCS is, in the meantime, a PoI, which needs to assign a positive weight according to its information priority.

In post-disaster scenarios, it is also important to consider the potential hazards and obstacles in the environment. In other words, it might not hold that the direct path between each node pair is viable. Therefore, the flying environment is modeled by a graph $G = (\mathcal{K}, E)$, where the edge set $E = \{(k_1, k_2) | k_1 \in \mathcal{K}, k_2 \in \mathcal{K}\}$ indicates the inter-node reachability.

We assume that time is slotted with index $t \in \{0, 1, \dots, T\}$. Without loss of generality, we assume at time slot 0, the UAV is at the GCS. Subsequently, at a time slot, if the UAV is at a node k_1 , the UAV can move to any of its neighboring nodes k_2 (including node k_1), which should be connected to node k_1 , after a fixed number of time slots via the edge (k_1, k_2) . For each edge $(k_1, k_2) \in E$, the UAV's travel time over the edge is denoted by $d(k_1, k_2) \in \mathbb{Z}^+$, and the energy cost for traversing this edge is denoted by $c(k_1, k_2) > 0$. For consistency, we assume that $d(k_1, k_2) = \infty$ if $(k_1, k_2) \notin E$ and $d(k_1, k_2) = 0$ if $k_1 = k_2 \neq 0$. Note that $d(k_1, k_1) = 0$ means that the UAV can stay at PoI $k_1 \neq 0$, but $c(k_1, k_1) (k_1 \neq 0)$ is positive because the UAV needs energy for hovering. Also, note that $d(0, 0)$ is treated specially to reflect a different meaning, i.e., recharging time as per Section III-C. We assume that $c(0, 0) = 0$ to reflect that energy cost during recharging is negligible.

To ensure each PoI is reachable, it is assumed that the graph is connected. In other words, starting from any node $k_1 \in \mathcal{K}$,

the UAV can reach any other node $k_2 \in \mathcal{K} \setminus \{k_1\}$ by traversing edges defined in E .

B. Closed Route and Periodic Route

To timely monitor all PoIs, the UAV should travel on the graph G along a judiciously planned route, i.e., a sequence of nodes where every two neighbouring nodes should be connected by an edge. Formally, a *route* $\mathbf{X}$ can be denoted by $(x_1, x_2, \dots, x_R)$, where x_l is the l th node to visit for $l = 1, \dots, R$, and the graph constraint can be denoted by

$$(x_l, x_{l+1}) \in E, \forall l = 1, 2, \dots, R-1. \quad (1)$$

Initially, the UAV should start from the GCS, i.e., $x_1 = 0$. Furthermore, we define a *closed route* [5] as a route that starts from and ends at the GCS, i.e., $x_1 = x_R = 0$, and $x_i \neq 0, 1 < i < R$.

Note that a *closed route* may not cover all PoIs.

For ease of scheduling, we need to find a route that (1) starts from and ends at the GCS, (2) covers all PoIs at least once, and (3) can be executed repeatedly. Such a route is called a *periodic route*. Note that a period route may consist of multiple closed routes, i.e., the GCS may appear multiple times in a period route. We use $\mathbf{X} = (x_1, x_2, \dots, x_L)$, $L \geq R$, to denote a periodic route, where

$$x_1 = x_L = 0. \quad (2)$$

C. Energy Constraint

On a micro scale, the instantaneous power or energy consumption of a UAV can be influenced by factors such as velocity [40]. Conversely, at a macro scale, a UAV's overall energy consumption is predominantly determined by the flight distance or duration, as these factors dictate the total duration of energy usage throughout the mission. In the specific context of our study, which targets post-disaster scenarios, the energy consumed by a UAV is generally assumed to be proportional to the flight distance [41], [42] or flight time [43], [44]. Furthermore, when dissecting a flight operation into its constituent sub-processes, e.g., vertical movement, horizontal movement, and hovering, extensive field tests reveal that the energy expenditure during the majority of these sub-processes is indeed linearly related to either the flight distance or time [45]. This finding indicates that a linear energy consumption model is practical in various scenarios, given that a UAV's flight operation inherently comprises these fundamental sub-processes.

In this paper, we consider a general energy consumption model where the energy consumed by traversing any edge $(k_1, k_2) \in E$ is given by a function $c(k_1, k_2)$. Consequently, the linear energy model is a special case of ours when the energy consumption is proportional to the flight distance or the flight time between any two nodes. Note that when $k_1 = k_2$, $c(k_1, k_2)$ indicates the energy cost for hovering at that node.

The UAV is assumed to be constrained by a limited energy budget B . Before running out of energy, the UAV must return to the GCS for recharging. Here, recharging refers to operations like refueling, battery replacement, or UAV replacement. After recharging for a fixed number of time slots, denoted by t_c , the

remaining energy of the UAV becomes B again. We set $d(0, 0) = t_c$ to reflect the fixed charging time.

Formally, within a periodic route $\mathbf{X} = (x_1, x_2, \dots, x_L)$, the remaining energy b_l of the UAV at the moment it leaves node x_l can be recursively defined for $l = 1, \dots, L$ as follows

$$b_l = \begin{cases} B, & \text{if } x_l = 0 \text{ and } g_l = 1 \\ & \text{and } b_{l-1} - c(x_{l-1}, x_l) > 0, \\ b_{l-1} - c(x_{l-1}, x_l), & \text{otherwise} \end{cases} \quad (3)$$

where g_l is the recharging decision variable of whether to recharge the UAV ($g_l=1$) at the l th node or not. Clearly, g_l can be set to 1 if and only if $x_l = 0$. Note that in the definition the condition $b_{l-1} - c(x_{l-1}, x_l) > 0$ means that the UAV has enough energy to return to the GCS, i.e., it can finish a closed route within the periodic route.

We fix g_0 to be 1, i.e., the UAV is initially full of energy and always recharges after finishing a periodic route.¹

Under such a definition, the remaining energy is set to a negative value when the UAV does not have enough energy to move to the next node. As a consequence, the energy constraint given a periodic route $\mathbf{X}$ is

$$b_l \geq 0, \forall l = 1, \dots, L. \quad (4)$$

D. Age Metric

The information gathered by the UAV regarding each PoI should be fresh over time to enable emergency responders to make well-informed decisions. The information freshness is commonly measured by the AoI metric, defined as the time elapsed since the generation time of the latest information held by the information holder, i.e., the GCS.

Specifically, at a time slot t , we use a binary variable $c_k(t) \in \{0, 1\}$ to denote whether the UAV is at node k ($c_k(t) = 1$) or not. Then the latest time $m_k(t)$ that the UAV monitors the node k is $m_k(t) = \arg \max\{\tau < t | c_k(\tau) = 1\}$.

By convention, we assume a fresh start of the system, i.e., $c_k(0) = 1, \forall k \in \mathcal{K}$. Subsequently, the AoI $h_k(t)$ of node k at a time slot t is $h_k(t) = t - m_k(t)$,

where the delay for data sampling and wireless transmission is neglected compared to the long inter-node travel time. Given the AoI value $h(\mathbf{X})$ for each node at every time slot following a periodic route $\mathbf{X}$, an AoI function maps it to a scalar, where two commonly-used ones are:

- **Time-average AoI (AVE-AoI)**, the weighted sum of cumulative AoI averaged on time for all nodes. Let $H_{\text{AVE}}(h(\mathbf{X}))$ denote the AVE-AoI for the periodic route $\mathbf{X}$. Then, $H_{\text{AVE}}(h(\mathbf{X})) = \frac{1}{T} \sum_{k \in \mathcal{K}} \sum_{t=0}^T w_k h_k(t)$.
- **Maximum Weighted AoI (MAX-AoI)**, the maximum AoI of nodes weighted by their weights. Let $H_{\text{MAX}}(h(\mathbf{X}))$ denote the MAX-AoI for the periodic route $\mathbf{X}$. Then, $H_{\text{MAX}}(h(\mathbf{X})) = \max_{k \in \mathcal{K}, 0 < t < T} w_k h_k(t)$.

To differentiate from the AoI value $h(\mathbf{X})$, we will use the term AoI score to refer specifically to the value of the given metric for a periodic route.

¹This assumption is to facilitate discussion since if the energy budget B can support multiple periodic routes, we can combine these routes as one periodic route without having any impact on our algorithm design and analysis.

Remark 1: There are also other functions of AoI, e.g., the average peak AoI [4]. The average peak AoI for a node is determined solely by its average visiting frequency, which is calculated by dividing the duration stayed at the node by the total time horizon. Thus, the optimal policy for maximizing peak AoI is fairly straightforward: following the TSP route to visit all the nodes, where each edge is traversed only once to maximize the number of time slots assigned to visit nodes. Due to the simplicity, we ignore this metric in this paper.

E. Problem Statement and Hardness

To timely monitor all PoIs subject to the energy and graph constraint, this paper investigates the AoI-minimizing UAV PoI-monitoring (AMUPM) problem, defined as follows.

Problem 1 (AMUPM Problem): Given graph $G = (\mathcal{K}, E)$, the weight of all nodes $w = (w_0, w_1, \dots, w_k)$, the traveling time $d(k_1, k_2)$ and energy cost $c(k_1, k_2)$ for each edge $(k_1, k_2) \in E$, energy budget B , and the AoI function H , the problem is to find a periodic route $\mathbf{X}$ and a recharging schedule $\mathbf{g} = (g_1, \dots, g_L)$ to minimize the AoI score $\lim_{T \rightarrow \infty} H(h(\mathbf{X}))$ subject to constraints (1), (2), and (4). Formally, let $(\mathbf{X}^*, \mathbf{g}^*)$ be the optimal periodic route and recharging schedule, the problem can be formulated as follows:

$$(\mathbf{X}^*, \mathbf{g}^*) = \underset{(\mathbf{X}, \mathbf{g})}{\operatorname{argmin}} \lim_{T \rightarrow \infty} H(h(\mathbf{X})), \quad (5a)$$

$$\text{s.t. (1), (2), (4)} \quad (5b)$$

Note that we should consider a theoretically infinite time horizon even if the periodic route is conducted repeatedly. This is because we do not know in advance whether or not AoI changes follow a periodic pattern and even so, the cycle of AoI changes may not align with the cycle of the route.

Next, we show that AMUPM is NP-hard for both age metrics.

Lemma 1 (NP-Hardness): The AMUPM problem is NP-hard when the given metric is AVE-AoI or MAX-AoI.

Proof: When the given metric is AVE-AoI, we consider the special case where the energy cost of each edge is zero and the distance between every two connected nodes is the same. Our problem is thus identical to the problem that minimizes the AVE-AoI on a unit-distance graph without energy constraint [4]. Consequently, by the similar argument in the proof of Theorem 4 in [4], our problem under this special case is NP-hard. Therefore, the more general problem is also NP-hard.

When the given metric is MAX-AoI, the min-max latency walk problem defined in [5] is a special case of our problem where the energy cost of each edge is zero, and the graph is undirected and complete. Since the min-max latency walk problem is NP-hard, our problem under the MAX-AoI metric is also NP-hard.

IV. THE LI2 ALGORITHM

As explained in Section II, DRL has become a go-to approach to tackling complex combinatorial optimization problems. It is worth noting that no effective approximation algorithm has been developed to solve our problem at hand. Although approximation algorithms have been designed for some special cases of

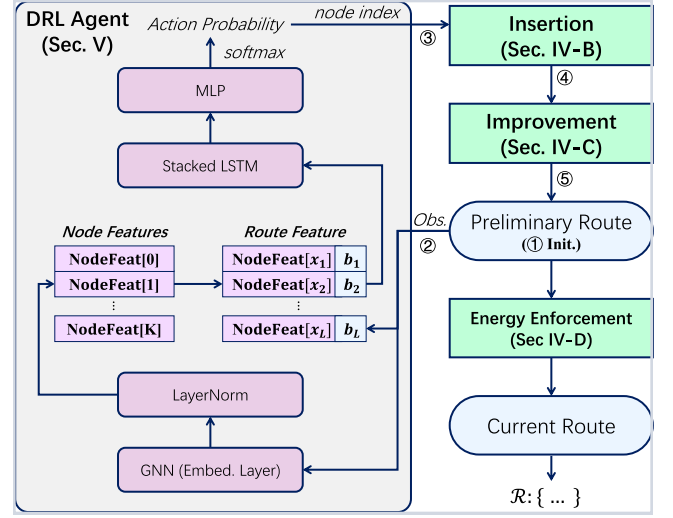


Fig. 2. The LI2 algorithm.

our problem, extending them to the general case is challenging. For example, [4] models the travelling on a unit-distance graph as status evolving on a Markov chain to derive AVE-AoI under the randomized walking policy in the long term. In contrast, deriving AVE-AoI under our problem is infeasible using this approach due to the different travel times on edges.

In this regard, metaheuristic optimization algorithms, which improve candidate solutions through iterative processes, have proven particularly effective in solving classical routing problems such as the TSP [46]. In fact, the TSP can be seen as a special case of our problem where there is no energy constraint and the given age metric is unweighted MAX-AoI [5]. At a very high level, our proposed LI2 algorithm follows the same iterative approach that generates a set of candidate solutions over time and returns the best one.

A. Overview

Fig. 2 and Algorithm 1 illustrates the overall framework of LI2. It depends on a DRL agent (detailed in Section V), as shown in the grey box of Fig. 2, to select a suitable node for the next iteration.

In typical metaheuristics, the solution is improved by applying simple operations repeatedly. However, directly modifying a periodic route in our problem can create routes that may violate the energy constraint. As a remedy, LI2 maintains a route that adheres to the graph constraint but not necessarily the energy constraint, referred to as *preliminary route*. Subsequently, the preliminary route undergoes modification through the *Energy Enforcement* module (detailed in Section IV-D) to produce a route that conforms to both constraints, termed the *current route*. The AoI score is then computed based on this current route, and the identical AoI score is concurrently designated as the AoI score of the preliminary route.

Fig. 2 and Algorithm 1 illustrates the overall framework of LI2. It depends on a DRL agent, shown in the grey box of Fig. 2

Algorithm 1: LI2.

```

1  $X^* \leftarrow$  Initialize the current route;
2  $\mathcal{R} \leftarrow \{EnergyEnforcement(X^*)\}$ ;
3 for  $n = 1, \dots, R$  do
4    $p \leftarrow$  The DRL agent selects a node;
5    $X^*, i \leftarrow Insertion(X^*, p)$ ;
6    $OPs \leftarrow \{op_1, op_2, \dots\}$ ;
7    $OPs' \leftarrow OPs$ ;
8   while  $OPs' \neq \emptyset$  do
9      $op \leftarrow$  Draw an operator randomly from  $OPs'$ ;
10     $OPs' \leftarrow OPs' \setminus \{op\}$ ;
11     $X' \leftarrow op(X^*, i)$ ;
12    if  $H(X') < H(X^*)$  then
13       $X^* \leftarrow X'$ ;
14       $OPs' \leftarrow OPs$ ;
15    end if
16  end while
17   $\mathcal{R} \leftarrow \mathcal{R} \cup \{X^*\}$ ;
18 end for
19 return  $\arg \max_{X \in \mathcal{R}} H(X)$ ;

```

and whose details will be disclosed in Section V, to select a suitable PoI for the next round of iteration. LI2 works as follows.

- 1) *Step 1: (Initialization)* LI2 finds an initial feasible route that encompasses each PoI. This route can be found with existing TSP algorithms such as that in [46]. We call this route as *current route*. Then the current route is validated via the *Energy Enforcement* module (Section IV-D), which checks if the route is feasible with energy budget B . If yes, this route remains unchanged. If not, *Energy Enforcement* revises the route to make it energy feasible.² The output route is called *feasible route* and is inserted into a candidate route set $\mathcal{R}$.
- 2) *Step 2:* The DRL agent selects a node, which can be a PoI or the GCS.
- 3) *Step 3:* The selected node and the *current route* are input to the *Insertion* module (Section IV-B), which inserts the selected node into the best position in the *current route*. Then the *current route* is replaced with the route output from *Insertion*.
- 4) *Step 4:* The current route is input into the *Improvement* module (Section IV-C), which aims to improve the AoI score on the current route with operations explained in Section IV-C. The improved route is then injected into the *Energy Enforcement* module to become an energy-feasible route, which is inserted into $\mathcal{R}$.
- 5) *Step 5:* Repeat from *Step 2* for R times where R is a predefined threshold. Here, R plays as a tuning knob to balance the quality of the final output and the running time of LI2.

²If there is at least one energy feasible route in the original graph, the *Energy Enforcement* module can always return an energy feasible path.

- 6) *Step 6:* Among the R feasible routes in the candidate route set $\mathcal{R}$, select the route with the minimum AoI score as the final output of LI2.

We point out that in each iteration above (*Steps 2-5*) the *current route*, after *Insertion* and *Improvement*, might not become better in terms of the AoI score. But this is not critical, because we keep generating more feasible routes, which with a good chance improve the *current route*.

B. Insertion

This module inserts a node p , determined by the DRL agent, into a given route. Consider two consecutive nodes v and u in a given route: If $(v, p) \in E$ and $(p, u) \in E$, then the feasible positions for inserting p are before v , after u , or between v and u ; If $(v, p) \in E$ but $(p, u) \notin E$, the feasible position for inserting p is before v ; If $(u, p) \in E$ but $(v, p) \notin E$, the feasible position for inserting p is after v ; Otherwise, there are no feasible positions for inserting p adjacent to u or v .

Since a route is ascribed by a sequence of nodes, *Insertion* enumerates all feasible positions for p according to the above procedure and inserts p into the feasible position that leads to the smallest AoI score. It is worth mentioning that the route output *Insertion* is not necessarily better than the current route. This necessitates the *Improvement* step introduced below.

C. Improvement

As shown in Fig. 2 and Line 8~16 in Algorithm 1, the *Improvement* module consists of a random scheduler and a set of operators. We first introduce how the random scheduler works. It repeatedly invokes one of the three operators with uniform probability while keeping track of so far the best route X^* (in terms of the AoI score). In each round, once an operator is chosen, the selected operator is applied to the best route X^* to generate some new routes by executing the predefined operation at different positions respectively. If the operator yields a strictly better route X' , the scheduler remembers this route, sets $X^* \leftarrow X'$, and continues for the next round. Otherwise, the scheduler chooses another operator that has not been used in the current round and repeats the above procedure. If no such operators are left in the current round, the *Improvement* module terminates and returns X^* .

As illustrated in Fig. 3, the operators include:

- *Reconnect*: This operator removes two different edges (k_1, k_2) and (k_3, k_4) in the given route and reconnects (k_1, k_3) and (k_2, k_4) , as long as the reconnection is feasible according to the network topology. This operator is also named 2opt in some metaheuristics [37] for TSP;
- *Exchange*: This operator first chooses two sub-routes, i.e., sub-sequences of the given route that have an equal number of nodes. Then, the operator exchanges the positions of these two sub-routes, as long as the exchange is feasible according to the graph topology. The node count of the sub-route is given as a parameter $C_{exchange}$;
- *Relocate*: This operator chooses a sub-route of the given node count $C_{relocate}$. Then it extracts this sub-route and

Algorithm 2: Operator Reconnect.

```

1 Operator Reconnect ( $X, i$ ):
2    $X^* \leftarrow X$ ;
3   for  $m = i - r_L, i - r_L + 1, \dots, i + r_R$  do
4      $m = m \bmod |X|$ ;
5     for  $\Delta = 2, 3, \dots, M + 1$  do
6        $n \leftarrow (m + \Delta) \bmod |X|$ ;
7       if  $m > n$  then
8          $m, n \leftarrow n, m$ ;
9       end if
10       $X^{mid} \leftarrow$  Reverse the order of nodes in
         $X[m : n]$ ;
11       $X' \leftarrow X[:m] \# X^{mid} \# X[n:]$ ;
12      if  $X'$  meets the graph constraint then
13         $X' \leftarrow \text{EnergyEnforcement}(X')$ ;
14        if  $H(X') < H(X^*)$  then
15           $X^* \leftarrow X'$ ;
16        end if
17      else
18        continue;
19      end if
20    end for
21  end for
22  return  $X^*$ ;
23 end

```

Algorithm 3: Operator Exchange.

```

1 Operator Exchange ( $X, i$ ):
2    $X^* \leftarrow X$ ;
3   for  $m = i - r_L, i - r_L + 1, \dots, i + r_R$  do
4      $m = m \bmod |X|$ ;
5     for
         $n = m + C_{exchange}, \dots, m + C_{exchange} + M - 1$ 
      do
6        $X' \leftarrow X^*$ ;
7       for  $\Delta = 0, \dots, M - 1$  do
8          $k_1, k_2 \leftarrow (m + \Delta) \bmod |X|, (n + \Delta) \bmod |X|$ ;
9          $X'[k_1], X'[k_2] \leftarrow X'[k_2], X'[k_1]$ ;
10      end for
11      if  $X'$  meets the graph constraint then
12         $X' \leftarrow \text{EnergyEnforcement}(X')$ ;
13        if  $H(X') < H(X^*)$  then
14           $X^* \leftarrow X'$ ;
15        end if
16      else
17        continue;
18      end if
19    end for
20  end for
21  return  $X^*$ ;
22 end

```

Algorithm 4: Operator Relocate.

```

1 Operator Relocate ( $X, i$ ):
2    $X^* \leftarrow X$ ;
3   for  $m = i - r_L, i - r_L + 1, \dots, i + r_R$  do
4      $X'_1 \leftarrow$ 
         $(X^*[m \bmod |X|], \dots, X^*[(m + M - 1) \bmod |X|])$ ;
5      $X'_2 \leftarrow$  Exclude  $X'_1$  from  $X^*$ ;
6     for  $\Delta = 1, \dots, M$  do
7        $n \leftarrow (m + \Delta) \bmod (|X'_2| + 1)$ ;
8        $X' \leftarrow X'_2[:n] \# X'_1 \# X'_2[n:]$ ;
9       if  $X'$  meets the graph constraint then
10         $X' \leftarrow \text{EnergyEnforcement}(X')$ ;
11        if  $H(X') < H(X^*)$  then
12           $X^* \leftarrow X'$ ;
13        end if
14      else
15        continue;
16      end if
17    end for
18  end for
19  return  $X^*$ ;
20 end

```

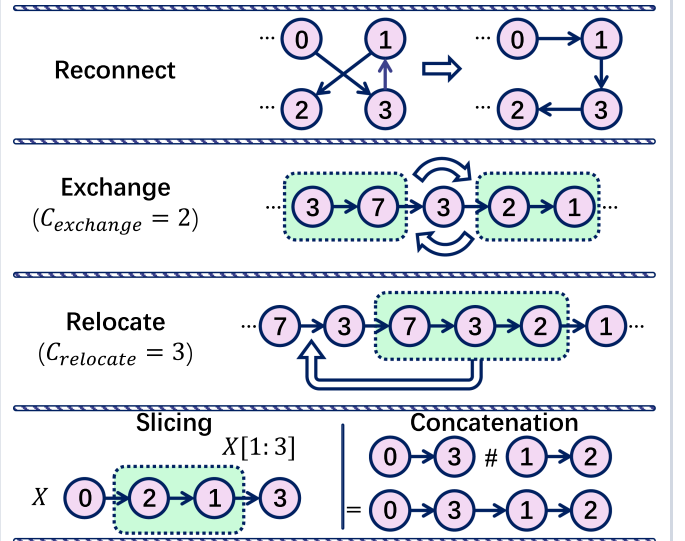


Fig. 3. Operations in *Improvement* (the top three figures) and operations in *Energy Enforcement* (the bottom figure).

re-inserts it into a different position as long as the re-insert meets the graph constraint.

Note that there are extra parameters, $C_{exchange}$ and $C_{relocate}$, in the Exchange and Relocate operators, respectively. Similar to [39], we instantiate each operator with different parameters, and each instance is assigned the same sampling probability.

D. Energy Enforcement

In the AMUPM problem, a recharging decision variable is needed to indicate whether to recharge the UAV at the GCS. The *Energy Enforcement* module makes the recharging decisions to

ensure a given route, after modification if necessary, can be supported with the energy budget.

Given a route $\mathbf{X}^I$, *Energy Enforcement* checks whether $\mathbf{X}^I$ is energy-feasible or not. If yes, *Energy Enforcement* does nothing and returns $\mathbf{X}^I$ immediately. If not, it forces the UAV to fly to the GCS for recharging. Specifically, for node $x_l^I, l = 2, \dots, |\mathbf{X}^I| - 1$, if the remaining energy when UAV leaves this node is not enough for it to reach its next target x_{l+1}^I , or after reaching x_{l+1}^I the remaining energy could not support it to fly to the GCS along the shortest route, the UAV should fly to the GCS along the shortest route before flying to x_{l+1}^I .

Following the above idea, *Energy Enforcement* first calculates the shortest route, denoted by $\mathbf{X}^S(k_1, k_2) = (k_1, \dots, k_2)$ in terms of the energy consumption, between every two nodes $k_1, k_2 \in \mathcal{K}$ and the cumulative energy cost $c^S(k_1, k_2)$ along this route. For every two consecutively-visited nodes (x_l^I, x_{l+1}^I) where $l = 2, \dots, |\mathbf{X}^I| - 1$, if $b_{l+1}^I < c^S(x_{l+1}^I, 0)$, add the following sub-route $\mathbf{X}^S(x_l^I, 0)[1:] \# \mathbf{X}^S(0, x_{l+1}^I)[1:]$ between x_l^I and x_{l+1}^I , where $\#$ meaning concatenation and $[:]$ is the slicing operator. Note that the default values of the first and second parameters of the slicing operator are the starting and ending positions, respectively, and $\mathbf{X}[i]$ simply indicates the i th node in the route $\mathbf{X}$. Fig. 3 illustrates an example of the above concatenation and slicing operations. The resulting route is denoted by $\mathbf{X}^{II}$.

With the above operations, *Energy Enforcement* ensures that the resulting route meets both energy and graph constraints, which will be formulated in Section IV-F.

E. Micro Improvement

In our implementation, we adopt two strategies to further improve LI2:

1) *Initial Route Selection*: The initial route (*Step 1*) fed to LI2 should be reasonably good. We utilize the TSP route, which visits each node once with approximately the minimum cumulative traveling distance, as an initial route. Alternatively, if no Hamilton cycle is available, we relax the condition that each node is visited exactly once and find a route with approximately the minimum cumulative traveling distance.

2) *Speed up Improvement*: If we simply let *Improvement* enumerate all possible changes to the current route, we might suffer from a scalability issue. As a remedy, the *Improvement* receives the insertion position from the *Insertion* module, and it restricts route changes within a smaller range.

In more detail, all three *Improvement* operators work as follows: 1) Select a sub-route in the current route; 2) Choose a position in the current route to conduct the operation. To shrink the operation range, it is restricted that at most $\lceil D/2 \rceil$ nodes exist between the first node of the sub-route selected in the first step and the inserted position, where D is referred to as the operation diameter. Also, we impose the constraint that at most M nodes exist between the position chosen in the second step and the sub-route selected in the first step. The specific implementation of the **Reconnect**, **Exchange**, and **Relocate** operator, incorporating this strategy, is presented in

Algorithms 2, 3, and 4, respectively, where we define

$$r_L = \left\lceil \frac{D}{2} \right\rceil, r_R = D - r_L - 1, \quad (6)$$

to represent the range of the first node in the selected sub-route.

This not only accelerates the *Improvement* phase but also attains a performance comparable to that of global enumeration. The rationale behind this is that these positions are most likely the ones that will be operated on when enumerating all possible positions because they are impacted the most by the *Insertion* in the previous iteration.

F. Feasibility

To formulate the feasibility of the LI2 algorithm, we first present the following lemma.

Lemma 2: If the route $\mathbf{X}^I$ fed to the *Energy Enforcement* module has met the graph constraint and the AMUPM is feasible to be solved, then the route $\mathbf{X}^{II}$ outputted from the *Energy Enforcement* module would be a feasible solution.

Proof: We first show that a feasible solution exists for the AMUPM problem if and only if the cumulative energy cost along the shortest route from any PoI to the GCS is no larger than $B/2$, i.e., $c^S(k, 0) \leq B/2, \forall k \in \mathcal{K}$ by respectively proving the necessity and sufficiency: 1) Assume that a feasible route $\mathbf{X}$ exists while a PoI $k' \in \mathcal{K}$ satisfying $c^S(k', 0) > B/2$ exists. Then, the cumulative energy consumption of consecutively visiting the GCS via the PoI k' when conducting the route³ would exceed the energy budget B since it is bounded lower by $2c^S(k', 0)$ that is greater than B , which contradicts the feasibility of the route $\mathbf{X}$ and completes the proof of the sufficiency; 2) Assume that $c^S(k, 0) \leq B$ holds for any $k \in \mathcal{K}$, then we can easily construct a route $\mathbf{X} = \mathbf{X}^S(0, 1) \# \mathbf{X}^S(1, 0) \# \mathbf{X}^S(0, 2) \# \mathbf{X}^S(2, 0) \# \dots$ where before visiting each PoI the UAV returns for the GCS for recharging along the route with the minimum energy cost. Such a route is feasible for the energy constraint and the graph constraint (as the shortest route exists for each node pair due to the global connectivity of the graph), thus proving the necessity.

Next, we show that if the route $\mathbf{X}^I$ is feasible for the graph constraint and $c^S(k, 0) \leq B/2, \forall k \in \mathcal{K}$, the route $\mathbf{X}^{II}$ is feasible. Recall that the route $\mathbf{X}^{II}$ is built on the route $\mathbf{X}^I$ by examining the whether the condition $b_{l+1}^I < c^S(x_{l+1}^I, 0)$ holds for every two consecutively-visited nodes (x_l^I, x_{l+1}^I) .

If the condition is validated to be false, then it holds that $b_{l+1}^I \geq c^S(x_{l+1}^I, 0)$, which indicates that $b_{l+1}^I \geq 0$. Furthermore, by definition in (3) $b_l^I \geq 0$ also holds.

Then we consider the case where the condition is validated to be true, i.e., $b_{l+1}^I < c^S(x_{l+1}^I, 0)$. In this case, by the checking result for node pair (x_{l-1}^I, x_l^I) it holds that $b_l^I \geq c^S(x_l^I, 0)$. This indicates that the remaining energy of the nodes in the first part $\mathbf{X}^S(x_l^I, 0)[1:]$ of the sub-route added to the route $\mathbf{X}^{II}$ outputted by the *Energy Enforcement* module would be non-negative. After recharging at the GCS, the remaining energy of the UAV turns

³Such a sub-route should exist since every PoI should be visited at least once in a periodic route.

to be B again. Subsequently, as $c^S(0, x_{l+1}) \leq B/2$, the remaining energy of the nodes in the second part $X^S(0, x_{l+1})[1:]$ of the newly-added sub-route is also non-negative. More importantly, $c^S(0, x_{l+1}) \leq B/2$ further guarantees that $b_{l+1}^I \geq B - B/2 = B/2 \geq c^S(x_{l+1}, 0)$, which enables the analysis above to restart from b_{l+1}^I and be applied to the remaining nodes.

Consequently, the remaining energy at each node in the route X^{II} is non-negative, indicating that the energy constraint has been met. Also, as both X^I and the modification that leads it to X^{II} adheres to the graph constraint, the proof completes.

This immediately results in the following theorem.

Lemma 3 (Feasibility of LI2 Algorithm): If the initial route X_0 has met the graph constraint and the AMUPM is feasible to be solved, then the solution returned by the LI2 algorithm adheres to both the graph and energy constraint.

Proof: Recall that all routes in the set $\mathcal{R}$ are outputted from the *Energy Enforcement* module, and every route fed to the *Energy Enforcement* module has been validated to be feasible for the graph constraint. Then by Lemma 2 it holds that all routes in $\mathcal{R}$ are feasible for both graph and energy constraints. Consequently, as the route returned by the LI2 algorithm is selected from the set $\mathcal{R}$, the outputted route of the LI2 algorithm is also feasible for both constraints.

G. Time Complexity

The time complexity of each operator is as follows.

Theorem 1 (Complexity): Denoted by L the number of nodes in the given route, the time complexity of **Reconnect**, **Exchange**, and **Relocate** operator is $O(DMK(L+1)^2)$.

Proof: Each operator employs a double nested for loop to enumerate possible changes to the given route. The outer loop executes D times, and for each iteration of the outer loop, the inner loop executes M times. Hence, for each given route, an operator would generate $O(DM)$ new routes.

In the inner loop: 1) The time complexity of constructing each new route is $O(L+1)$; 2) Checking the viability between every two nodes that are neighboring in the new route, the number of pairs for which is $L+1$, would be sufficient to validate its feasibility for the graph constraint;⁴ 3) The time complexity for the *Energy Enforcement* module to process the given route is $O(L+1)$ as it checks every edge once; 4) The time complexity for calculating the AoI score is bounded above by $O(K(L+1)^2)$. Having processed by the *Energy Enforcement* module, the node count of the resulting route is bounded by $O((L+1)^2)$, where the worst case is the UAV has to return to the GCS for recharging via many nodes before visiting almost every node. In the resulting route, there are K PoIs. For each PoI k , suppose it appears n_k times in the new route, then record the first n_k timestamps the UAV visits this PoI k along this route can fully characterize the long-term AoI score of PoI k under this route, since it would be conducted periodically. Subsequently,

⁴Indeed, checking the node pairs that are impacted by the operator would be enough to guarantee the validation result is correct, which results in $O(1)$ time complexity.

calculating the gap between each two consecutive visits results in the peak values of the AoI of PoI k in one period. Then, the long-term AoI can be obtained by calculating the AoI score in this period. The time complexity of the process above is $O((L+1)^2)$, since it can be done by enumerating each edge in the route limited times.

Consequently, the overall time complexity for an operator is $O(DM \cdot K(L+1)^2)$, which completes the proof.

In practice, both D and M are set to a small constant. Moreover, the worst case that leads to a node count of $O((L+1)^2)$ for the route outputted by the *Energy Enforcement* module, i.e., the UAV has to return to the GCS for recharging via the path across almost all PoIs before visiting each PoI in the route, seldom occurs. Therefore, the time complexity of an operator is dominated by $O(K(L+1))$ for the most time. Meanwhile, $L+1$ is bounded above by $O(K+R)$ if the node count of the initial route is $O(K)$, i.e., when most PoIs are visited only once following the initial route. Thus, empirically the time complexity of an operator is more likely to be $O(K(K+R))$.

The time complexity of the LI2 algorithm is influenced by the frequency of operator calls, which is determined by the initial route and the random operator scheduler. In cases where the initial route is of high quality, there is limited room for improvement, resulting in fewer operator calls. Consequently, the LI2 algorithm is expected to conclude more quickly in later stages. Conversely, during the early phase, the algorithm may require a comparatively longer running time, and this is a trade-off for improving the quality of the route. To quantitatively evaluate the time required to solve a problem instance, we conducted numerical experiments as described in Section VI. The results of these experiments demonstrate the practicality of our solution for disaster response scenarios.

H. Discussion

1) Adaption to Environment Changes: The dynamic nature of disaster zones, where conditions can change rapidly, requires LI2 to adapt following real-time environment information. For example, a previously impassable path due to fire and smoke might become viable once the fire is extinguished, while new hazards may render other paths unviable. The LI2 algorithm can be applied to find an AoI-efficient periodic route based on the newest environment information. To be more specific, if the environmental changes are minor, the algorithm can adjust the existing route by generating and evaluating multiple alternative routes to find the optimal one for the current situation. In cases where significant changes occur, the iteration can start from an initial route where each PoI is visited once. Additionally, if certain edges in the original route become infeasible, the algorithm can replace these edges with the shortest paths between the affected nodes, based on the current environment. This ensures that the route remains efficient and responsive to the evolving conditions of the disaster zone.

2) Optimality and Convergence: Due to the black-box nature of the DRL agent used in the LI2 algorithm, it is challenging to derive the exact approximation ratio that bounds the worst-case

performance relative to the global-optimal algorithm for any given problem instance. However, LI2 is an iterative algorithm that generates multiple routes and selects the best one among them. Consequently, the route produced by LI2 is guaranteed to be no worse than the initially provided route. In other words, if the approximation ratio of the algorithm that generates the initial route is γ , the LI2 algorithm will converge to a route that is γ -optimal, ensuring that the approximation ratio of LI2 is no worse than γ .

V. THE DRL AGENT

In this section, we elaborate on the operation and training of the DRL agent. The execution process of the DRL agent proceeds as follows: First, information extracted from the current route, known as the observation, is forwarded to a deep neural network, referred to as the policy network. The policy network processes this observation to produce an output, based on which the DRL agent selects an action.

The training process optimizes the policy network to maximize the cumulative reward obtained from the agent's interactions with the environment. This is achieved by collecting trajectories, each consisting of an observation, the corresponding action, and the resulting reward. The policy network's parameters are then updated using these collected trajectories.

Next, we will detail the DRL agent's key components: the observation, action, policy network, reward, and learning algorithm.

A. Observation

The observation consists of three types of information that will be used by the policy network, including the edge weights of the (PoI/GCS) graph, the energy status, and the node status. The weight of each edge is defined as the reciprocal of the travelling time along the edge, indicating that greater attention should be given to nodes with closer positions. The energy status refers to b_l , as defined in (3), for each node x_l in the preliminary route when executing the current route.

We use a polar coordinate system to determine the relative positional relationships between nodes. For this purpose, we define a "mean point" as the pole, which is a virtual point whose x and y coordinates are calculated as the mean values of all the nodes' x -axis and y -axis values, respectively. The right horizontal direction is the reference direction of the polar coordinate. With this polar coordinate system defined, the node status is recorded in a matrix, where each row represents a node of the PoI/GCS network and each row includes five pieces of information: (1) the distance from the node to the pole of the polar coordinate system, (2) the angle of the node in the polar coordinate system, (3) the weight of the node (same as that defined in Section III-A), (4) the visiting frequency of the node under the current route, and (5) the minimum distance to travel from the node to the GCS.

Note that the environment is fully observable in our RL setting, i.e., the state space is considered the same as the observation space.

B. Action

As illustrated in Algorithm 1, the DRL agent selects a node, which can either be a PoI or the GCS. Notably, the DRL's policy network does not directly produce the action. Instead, the policy network generates the sampling probability for each node, forming a probabilistic distribution. The final action is then determined by sampling from this distribution.

C. Policy Network

The node status and the edge weights are sent into a graph neural network (GNN) [47] as the embedding layer to extract node features. The GNN aggregates the observations of each node along the specified edge set. Since the edge set may vary over time, our policy network can adapt to dynamic disaster environments. The node features are then re-arranged based on the preliminary route and concatenated with the energy status to create the route feature.

Having obtained the route feature, we first perform layer normalization, which can stabilize the hidden status of neural networks and boost the training process [48]. After the layer normalization, the route feature is forwarded to multiple stacked LSTM [49] layers for further processing. The output of the last LSTM layer is a tensor with a fixed shape. Such a fixed shape allows the tensor to be processed by a multi-layer perception (MLP) followed by a softmax function to yield the sampling probability for each node/action.

D. Reward

The goal of the LI2 algorithm is to improve the AoI score of the route. Therefore, the DRL agent should be rewarded when the new route achieves a better AoI score after *Insertion*. Specifically, for each episode that begins with the initial route and ends when a predefined number of nodes have been inserted, we track the best AoI score achieved in the episode. Then, every time the DRL agent provides a node index, the AoI score of the new route after the *Insertion* and *Improvement* operations is compared to the current best AoI score. Consequently, if the AoI score of the new route is not lower, the reward of this action is 0. Otherwise, the reward of this action is set to the reduced AoI score.

Remark 2: Note that we do not penalize the DRL agent for failing to improve its current route. This is because the agent may be able to improve after inserting multiple nodes. Imposing an immediate penalty would discourage the agent from exploring potentially beneficial policies.

E. Learning Algorithm

Undoubtedly, the quality of the solution found by the DRL agent and its training time are both critical in a post-disaster context. Thus, we adopt the popular Proximal Policy Optimization (PPO) algorithm that strikes a good balance between the sample efficiency and the algorithm complexity [50] to train the DRL agent. Below, we briefly introduce the training process under PPO.

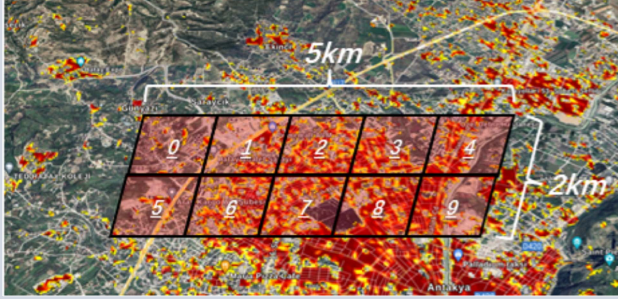


Fig. 4. Areas selected on the DPM for the Turkey earthquake, including regions that exhibit both concentrated and dispersed impacts of the disaster.

During training, the PPO algorithm collects a batch of trajectories with the current policy network and then updates the network parameters. This process repeats until a given number of rounds has been finished. Specifically, at the n th update, the PPO algorithm aims to maximize

$$J^{CLIP}(\theta) = \hat{\mathbb{E}}_n[\min(r_n(\theta)\hat{A}_n, \text{clip}(r_n(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_n)]. \quad (7)$$

In the above objective function: 1) θ is the set of network parameters to be optimized; 2) $r_n(\theta)$ denotes the ratio of probability $\pi_\theta(a_n|o_n)/\pi_{\theta_{old}}(a_n|o_n)$ at the n th step between the current policy π_θ and the policy with parameter θ_{old} before the update, where a_n and o_n are, respectively, the action and observation; 3) $\hat{A}_n$ is an estimator for the advantage function, which measures how good an action is (indicated by the reward received) compared to the expected value under the current observation. The advantage function is output by another neural network known as the critic. Typically, the critic has the same structure as the policy network but with minor modifications at the last layer to output a scalar; 4) $\text{clip}(r_n(\theta), 1 - \epsilon, 1 + \epsilon)$ returns the value of $r_n(\theta)$ if it falls in $[1 - \epsilon, 1 + \epsilon]$, $1 - \epsilon$ if $r_n(\theta) < 1 - \epsilon$, and $1 + \epsilon$ if $r_n(\theta) > 1 + \epsilon$. This clip operation is to avoid an excessively large update step and performance collapse, where ϵ is a hyperparameter. Finally, the PPO algorithm incorporates the objective function into the stochastic gradient ascent algorithm to update the network parameters. Refer to [50] for more details of PPO.

VI. EXPERIMENTAL EVALUATION

A. Algorithm Implementation and Parameter Setup

1) *DRL Agent*: Our DRL agent's policy network has a GNN output channel size of 16, two stacked LSTM layers, a hidden size for the LSTM network that is twice its input size, and two hidden layers in the MLP, each with a size of 128. We implement PPO using the Tianshou open-source DRL library [51]. We use most of the default learning hyperparameters in the library but decrease the learning rate to 0.0001 to prevent rapid convergence to local optimums and increase the discount factor to 0.99 to give enough credit to early *Insertion* actions.

We train the DRL agent for 20,000, 25,000, 30,000, and 35,000 steps when the number of PoIs is 10, 30, 70, and 100, respectively, using a batch size of 64. The training is performed

on a server with an Intel Xeon E5-2678 CPU and 4 RTX 3090 GPUs. We train the DRL agent five times with different random seeds for each problem instance, test it 20 times under each seed, and report the average performance.

2) *LI2*: The *Exchange* operator is instantiated triple times with $C_{exchange} = 1, 2, 3$. The *Relocate* operator is also instantiated with $C_{relocate} = 1, 2, 3$. The threshold R is set to 10, 15, 20, and 25 meters when the number of PoIs is 10, 30, 70, and 100, respectively. The operation diameter D is set to 9. The initial routes are solved with the Gurobi optimizer, the time limit of which is set to 1, 3, 7, and 10 when K is 10, 30, 70, and 100, respectively.

B. Comparison With DRL Baselines on Real Datasets

1) *Environment Setup*: To generate the positions and weights of PoIs, we utilize the open-source postdisaster analysis dataset provided by the ARIA collaboration [52]. More specifically, we rely on the damage proxy map (DPM), which employs heat values to indicate the severity of the damage, caused by the 2018 Sulawesi earthquake, the 2018 Woolsey fire, the 2023 Turkey earthquake, and the 2019 California fire. With this information, we generate 10 problem instances when the number of PoIs is 10, 30, 70, and 100, respectively. We refer to each of these instances as a dataset.

The process of generating a dataset with K PoIs is as follows: 1) We select ten $1 \text{ km} \times 1 \text{ km}$ areas from each DPM and record the heat value of each pixel within the area, an example of which is shown in Fig. 4. 2) Each pixel is mapped to a point in this area, and the pixel's heat value is set to be the point's weight; 3) The K-means algorithm is applied to all points of an area to generate K clusters; 4) Each cluster is mapped to a PoI whose position is the cluster centroid and weight is the summation of all points' weights in this cluster; 5) We normalize each PoI's weight by dividing each weight by the maximum weight.

For each problem instance, we randomly draw a point within the area as the GCS, whose weight is set to 0. Having obtained the positions and weights of all nodes, we randomly put 10 to 30 obstacles⁵ into the area, and any edge that comes across any obstacle is forbidden to traverse. The flying speed of the UAV is set to 10m/s, then the travel time $d(k_1, k_2)$ between two viable nodes (k_1, k_2) is the Euclidean distance divided by this speed. Since energy cost is tightly coupled with flying time, we ignore the unit of energy and set the energy cost $c(k_1, k_2)$ of traversing an edge (k_1, k_2) to be the travel time $d(k_1, k_2)$ for $k_1 \neq k_2$. The energy budget is set to 1200, i.e., 20 minutes. Note that $c(k_1, k_2)$ is set to 1 for $k_1 = k_2$, indicating the energy consumption for hovering at the PoI. As different recharging techniques might be adopted at various scenarios, the recharging time t_c is independently sampled from $[10, 30]$ with uniform probability for each instance.

2) *Baselines*: We evaluate two types of DRL baselines covering all the mainstream DRL-based solutions for the AoI-driven UAV routing problem. The first type of baseline operates in

⁵To generate a random obstacle, we first draw a point within the area. Then, we randomly add 3 to 6 vertices that are 5m to 35m away from this point, and connect them in clockwise order.

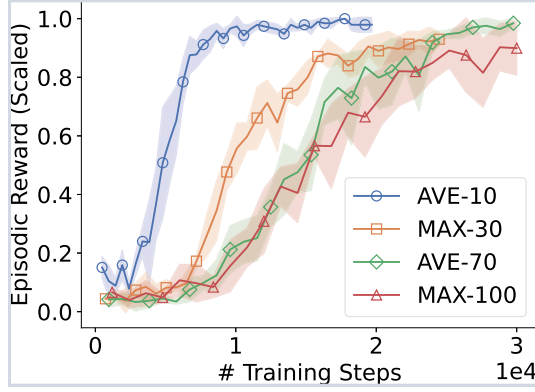


Fig. 5. Training curve. Mean episodic reward (solid line) and the standard deviation (shaded region) in our DRL agent's training process for 4 problem instances across 5 different seeds.

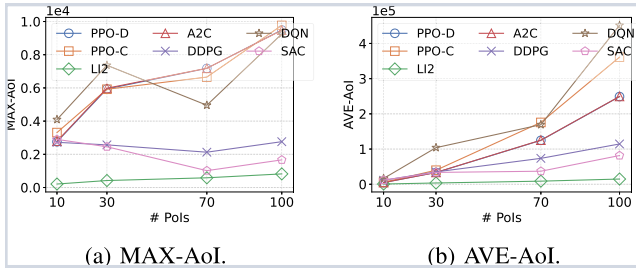


Fig. 6. AoI scores of our algorithm and DRL baselines.

discrete action space and learns the next node index. This type includes DQN [53], PPO [50], and A2C [54] RL algorithms for discrete action spaces. The second type of baseline operates in continuous action space and learns the next moving position. If the DRL baseline provides x and y coordinates for the UAV to move to, the UAV first calculates the nearest node to the point (x, y) subject to the graph constraint and then moves to that node. This type of baselines includes SAC [55], DDPG [56], and PPO [50] RL algorithms for continuous action spaces.

The above baselines only consider the graph constraint. To ensure the complete problem state is encoded, we adapt the observation to include node features of all nodes and the past R visited nodes. We also mask infeasible actions and use a weighted AoI value as the immediate reward. We use the same hyperparameters with our DRL agent while keeping others at their default values in Tianshou [51]. We set the number of steps per episode to $10R$ for the baselines.

3) *Convergence*: We present the training curve of our DRL agent, which records the AoI reduction every time the LI2 algorithm returns a periodic route, i.e., cumulative reward per episode w.r.t. the number of training steps. Fig. 5 displays the results of four problem instances, with the recorded rewards normalized to range $[0, 1]$ for comparability. The figure reveals that the episodic reward initially increases rapidly during training and then stabilizes, indicating convergence of our DRL agent. This trend is observed across most problem instances.

4) *AoI Performance*: The MAX-AoI and the AVE-AoI performance with LI2 and all baselines are shown in Fig. 6. The

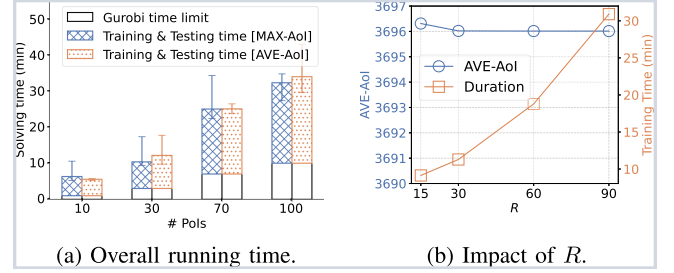


Fig. 7. (a) The overall running time of the LI2 algorithm to obtain the solution. (b) Impact of the threshold R when $K = 30$ and optimizing the AVE-AoI, including both the AoI score and the consumed training time.

results demonstrate that the average performance of LI2 outperforms all baselines at each dataset for AoI metrics. Specifically, compared to any DRL baseline over all problem instances, **LI2 reduces the MAX-AoI by at least 74.654% (SAC) and up to 92.066% (DQN). It also reduces the AVE-AoI by at least 82.910% (SAC) and up to 96.217% (DQN).**

As the number of PoIs increases to 10, 30, and 70, there is no observable upward trend in the MAX-AoI performance of DQN, DDPG, and SAC. This behavior is distinct from the pattern seen with the AVE-AoI metric. This discrepancy may be attributed to the fact that MAX-AoI is more sensitive to the travel route, as it logs the most adverse AoI encountered, while AVE-AoI maintains a record of average performance over time. Furthermore, these DRL baselines find certain instances particularly daunting when the value of K is 10 and 30, leading to exceedingly high MAX-AoI scores. For instance, the third instance is notably challenging when $K = 30$, as evidenced by DDPG and DQN, which register their highest MAX-AoI at this point. Similarly, the SAC algorithm's performance is second-worst for this instance. In stark contrast, **LI2 exhibits the least variance in both AoI metrics across all instances within each dataset, underscoring its robustness compared to all the DRL baselines.**

5) *Time Complexity*: To evaluate the overall time complexity of our solution, we track the overall running time for obtaining the final solution. As illustrated in Fig. 7(a), the overall running time includes 1) the time for generating the initial route, which is strictly bounded above by the Gurobi time limit in the figure, and 2) the time for training the DRL agent and testing. The mean overall running time for our LI2 algorithm is 5.912, 11.243, 24.970, and 33.044 minutes when K is 10, 30, 70, and 100, respectively. Even when dealing with 100 PoIs, the maximum overall running time is 38.742 minutes in our experiment. This timeframe aligns well with the critical nature of the response to disasters like earthquakes, where the first 24 hours post-event witness a survival rate exceeding 90%, and the golden period extends to 72 hours [57].

In comparison, an efficient heuristic algorithm for solving a constrained routing problem can take multiple hours to produce a solution with 100 nodes. For instance, the state-of-the-art heuristic algorithm LKH-3 requires approximately 13 hours to solve the Capacitated Vehicle Routing Problem [58], as tested on a comparable CPU (Xeon E5-2630). Additionally, akin to many

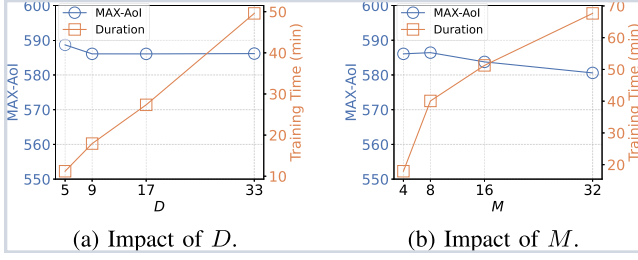


Fig. 8. Impact of the parameter D and M , including the resulting AoI score and the consumed training time, in speeding up *Improvement*, when $K = 70$ and optimizing the MAX-AoI.

heuristics, our solution allows for a trade-off between runtime and performance. For example, the training time can be reduced to obtain a solution more quickly in emergencies. In summary, relative to established heuristic routing algorithms, our proposed solution showcases a satisfactory running time, particularly when addressing scenarios with a substantial number of nodes.

6) *Impact of Threshold R* : To investigate the impact of the threshold R on the performance of LI2, we further vary R to be 30, 60, and 90 when $K = 30$ to optimize the AVE-AoI metric. To ensure the same number of training episodes as we train our DRL agent for 25000 steps when $R = 15$, the number of training steps is set to 50,000, 75,000, and 100,000, respectively, for $R = 30$, 60, and 90. The results in Fig. 7(b) show that the DRL agent with a larger value of R can find better periodic routes. However, the improvement is limited, considering the superlinear increase in training time. Specifically, the best MAX-AoI achieved by the setting of $R = 90$ is only reduced by 0.008% compared to our initial setting $R = 15$. This suggests that inserting 15 nodes is sufficient for LI2 to find efficient periodic routes for this dataset.

7) *Impact of D and M* : As depicted in Section IV-E, to speed up the *Improvement*, we restrict the operation range of each operator to the vicinity of the newly inserted node. The shrunk range is characterized by two parameters D and M . To validate this idea, we further test the impact, including the AoI score and the training time, of these two parameters. Specifically, when $K = 70$ and the objective is to optimize MAX-AoI, we set M to a fixed value of 4 and explore different values of D , namely 5, 9, 17, and 33, the results of which are shown in Fig. 8(a). Also, we fix D at 9 and vary M across 4, 8, 16, and 32, as reported in Fig. 8(b). We can see that increasing both parameters can lead to a better AoI score, but the improvement is limited compared to the extra time cost. To be specific, the MAX-AoI is reduced by at most 0.426% and 0.995% under the largest D and M , compared to that under our settings in Section VI-A, respectively. However, the consumed training time increases by 277.213% and 342.257%, respectively. These results suggest that the idea of restricting the operator to be operated in the vicinity of the newly inserted node can significantly improve the training speed while achieving satisfactory AoI performance.

8) *Impact of B* : To examine the impact of the energy budget on AoI performance, we vary the budget B across the set $\{400, 600, 800, 1000, 3000, 6000\}$, while considering two different numbers of PoIs, $K = 30$ and $K = 100$. We then measure the effects on both the AVE-AoI and MAX-AoI metrics. The

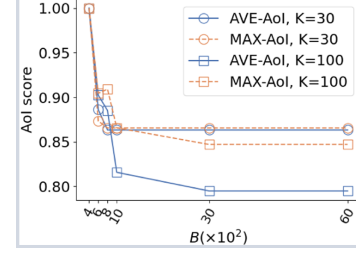


Fig. 9. Impact of energy budget B on the AoI score (normalized).

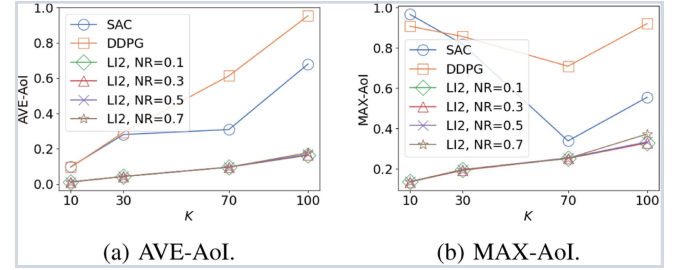


Fig. 10. Normalized AoI scores of our algorithm on the non-linear energy consumption model and DRL baselines.

results, presented in Fig. 9, reveal a consistent trend: an increase in B from 400 decreases AoI scores. However, this decline shows signs of diminishing returns with higher values of B . This phenomenon may be attributed to the fact that expanding the energy budget from a modest value significantly broadens the feasible action space within the constraints of the energy budget. Conversely, when the energy budget is sufficiently ample, the action space becomes expansive enough to achieve a desirable level of performance. Furthermore, the results indicate that the benefits of a higher energy budget are more pronounced in scenarios with a greater number of PoIs. Specifically, when $K = 100$, the enhancement in AVE-AoI surpasses 20%, while for $K = 30$, the enhancement remains within 15% across both AoI metrics.

9) *Impact of Energy Model*: Although we employ the linear energy consumption model (also widely used in UAV route planning), our LI2 algorithm can address a more complicated energy model since it takes a general, weighted graph as the input of energy consumed in travelling between PoIs. Considering that accurate energy consumption in practice can be dominated by the physical distance between the PoIs and impacted by other factors like wind disturbance, we further test the performance of our LI2 algorithm under the non-linear energy consumption model. Specifically, denoted by c the original energy consumed by travelling between two PoIs, it is reset to a random value within the range $[(1 - NR)c, (1 + NR)c]$, where NR is a scalar called noisy range. We evaluate LI2's performance when NR is 0.1, 0.3, 0.5, and 0.7, respectively. As illustrated in Fig. 10, results demonstrate that on both AoI metrics and under each setting of K , our LI2 algorithm consistently outperforms all other DRL baselines.⁶ Furthermore, the small variations in performance among the LI2 algorithms under various noisy range conditions

⁶For clarity, only the leading DRL baselines, i.e., DDPG and SAC, are plotted in the figure, as they consistently achieve the best AoI scores among all the DRL methods tested.

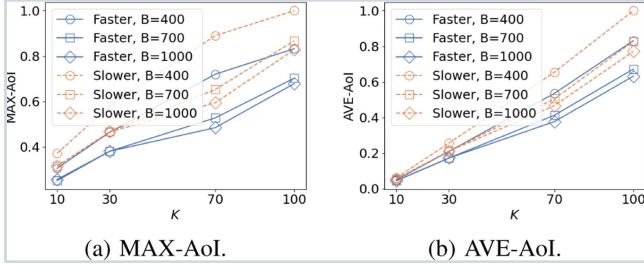


Fig. 11. Normalized performance of the LI2 algorithm under UAVs of different flying capabilities and energy efficiencies.

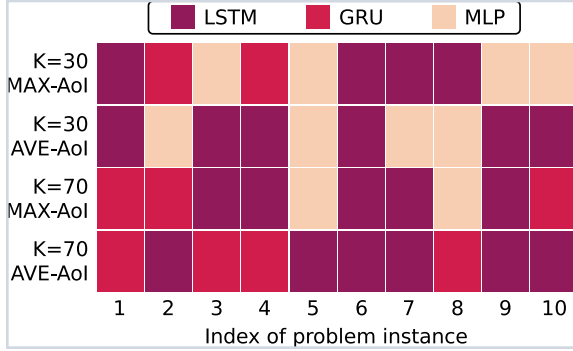


Fig. 12. Winner on AoI scores among LI2 algorithms with different neural networks for feature extraction.

demonstrate the algorithm's stability and adaptability. These findings further confirm that our LI2 algorithm can robustly accommodate non-linear energy consumption models.

10) *Impact of UAV Type*: To comprehensively evaluate the impact of UAV capabilities on AoI performance across diverse scenarios, we extend our analysis to account for variations in UAV flight characteristics. Specifically, we investigate the effects of different energy budgets—400, 700, and 1000—coupled with variations in flight speed. For the scenario with increased speed, we adjust the flight time required to traverse an edge by randomly scaling it to a value between 80% and 100% of the original time. Conversely, for the slower UAV scenario, we increase the flight time to a range of 100% to 120% of the original value. As shown in Fig. 11, we find a consistent trend: an increase in either flight speed or energy budget leads to a more pronounced reduction in AoI scores when the number of PoIs is greater.

11) *Impact of Neural Network*: To evaluate the influence of neural network architectures for feature extraction on AoI performance, we have substituted the stacked LSTM layer, as depicted in Fig. 2, with both stacked GRU and a four-layer Multilayer Perceptron (MLP) network. Then, we assess the performance of the LI2 algorithm under two different configurations of K , namely $K = 30$ and $K = 70$. The results, presented in Fig. 12, reveal that our framework exhibits compatibility with various neural network types, as both GRU-based and MLP-based feature extractors can deliver the best performance across both metrics. Furthermore, solutions utilizing recurrent neural networks (RNNs), including GRU or LSTM networks, tend to outperform those based on MLPs. Specifically, RNN-based

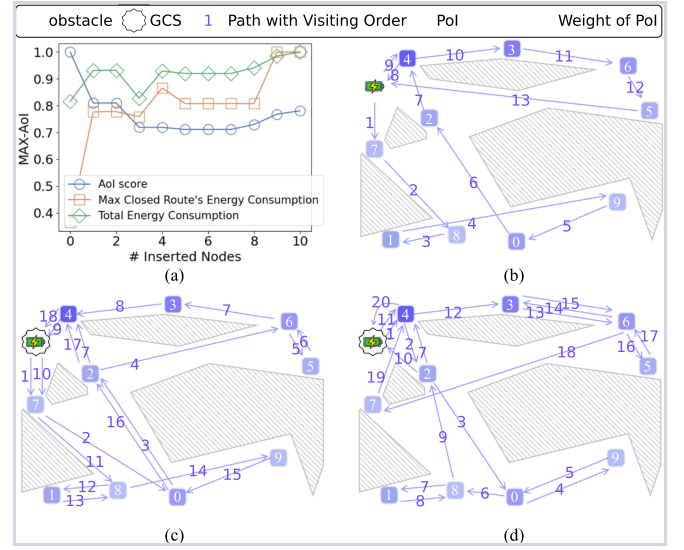


Fig. 13. A step-by-step demonstration of our LI2 algorithm: (a) The variation of AoI score and energy consumption of the generated route (normalized) with the number of inserted nodes; (b) Initial route; (c) Current route after inserting two PoI nodes; (d) Current route after inserting four PoI nodes.

solutions prevail in 75% of the instances, with LSTM-based solutions demonstrating the highest winning ratio. The superiority of RNN-based solutions becomes more pronounced as the number of PoIs increases. For instance, when $K = 70$, RNN-based solutions emerge as the winner in 90% of the instances, underscoring the enhanced performance of RNNs in handling larger-scale problems.

12) *a Step-By-Step Demonstration*: To illustrate the dynamic evolution of the generated route as new PoIs are incrementally incorporated, we provide a step-by-step demonstration in Fig. 13, specifically for the scenario where $K = 10$. The figure shows that the AoI score experiences an initial decline with the insertion of new PoIs during the early iterations of the LI2 algorithm, as depicted in Fig. 13(a). However, when the count of inserted PoIs surpasses seven, the AoI score begins to deteriorate following additional insertions. This deterioration is accompanied by a steep increase in both the periodic and longest closed routes' energy consumption, indicating that including an overly long closed route may not be advantageous for minimizing MAX-AoI. Fig. 13(c) to (d) visually represent the modifications to the current route output by the *Energy Enforcement* module after inserting 2 and 4 PoI nodes, respectively. The initial state features a single closed route, which is subsequently augmented by the addition of a second closed route to form the periodic route. Additionally, the frequency of visits to PoIs is influenced by their respective weights, with higher-weight PoIs being prioritized for more frequent visits.

C. Comparison With Approximation Algorithms

1) *Information Gathering (IG) Problem* [4]: The IG problem aims to optimize the AVE-AoI on a unit-distance graph without energy constraint. We implement two randomized traveling algorithms with proved approximation ratios, i.e., Fastest-Mixing

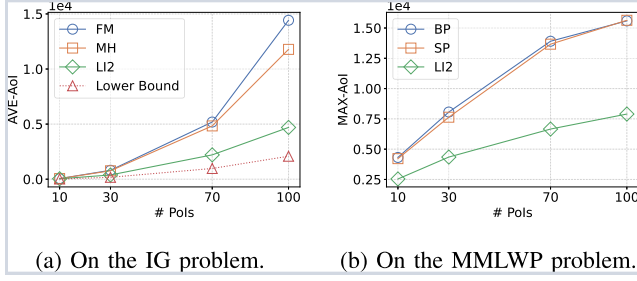


Fig. 14. Results compared to approximation algorithms.

(FM)⁷ and Metropolis-Hastings (MH) algorithms, which are respectively $\mathcal{H}$ -optimal⁸ for the AVE-AoI metric and global-optimal for the weighted-average peak AoI.

Following the simulation settings in [4], we generate 10 random geometric graphs with the same parameters and vary the number of PoIs from 10, 30, 70, to 100. The performance of FM, MH, and LI2 is compared with the globally lower bound (LB) of AVE-AoI, as illustrated in Fig. 14(a). We can see that the AVE-AoI for LI2 is at most a factor 2.3 away from the LB at any graph size in the test. Averaged on all graphs, **LI2 reduces the AVE-AoI by 57.989% and 64.218% compared to the MH and FM algorithms, respectively.**

2) *Min-Max Latency Walk (MMLW) Problem* [5]: This problem aims to optimize the maximum weighted latency of any node, i.e., the MAX-AoI, on a complete graph. We implement the two approximation algorithms, i.e., the brute partition (BP) and smart partition (SP) algorithms [5], whose approximation ratios are respectively $O(\log(\max_k w_k / \min_k w_k))$ and $O(\log K)$. We generate 10 instances with random PoI positions and weights and change the number of PoIs from 10, 30, 70, to 100. As illustrated in Fig. 14(b), LI2 shows superior performance compared to both the BP and SP algorithms at every graph scale. Averaged over all problem instances, **LI2 reduces the MAX-AoI by 48.794% and 47.907% compared to the BP and SP algorithms, respectively.**

D. Field Test

Conducting field tests in conditions akin to post-disaster scenarios poses two significant challenges: (1) procuring a city-wide drone flight permit from municipal authorities is often a daunting task, and (2) the considerable positioning error ($\sim 10\text{m}$) inherent in outdoor drones, primarily due to GPS limitations, results in inaccurate evaluation outcomes. To circumvent these issues, we use a Crazyflie 2.1 indoor drone. With the assistance of four SteamVR base stations and a Lighthouse deck mounted onto it, this model supports enhanced positioning accuracy ($\sim 10\text{ cm}$). *We leave city-wide outdoor tests as our future work.*

As illustrated in Fig. 15(a), we let the UAV visit 10 PoIs of different weights deployed within the test area to optimize the MAX-AoI, where the obstacle blocks some inter-PoI paths. Since the Crazyflie 2.1 drone has enough energy to support

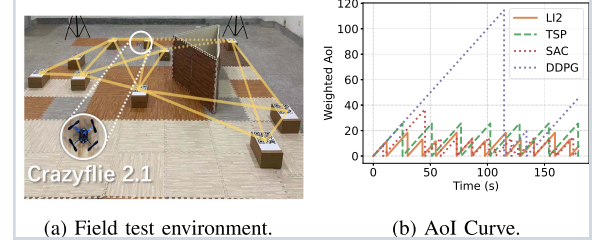


Fig. 15. (a) Field test environment. (b) AoI curve for the node leads to the MAX-AoI under the TSP waypoints.

hundreds of periodic routes, the energy constraint is not an issue in this indoor test. The UAV is given periodic routes in the form of waypoint sequences, respectively, provided by our LI2 algorithm, the TSP algorithm (which offers LI2's route as described in Section IV-E, and the leading DRL baselines SAC and DDPG that achieve the best AoI scores among all DRL baselines as shown in Fig. 6. Subsequently, the UAV embarks on the routes following the designated waypoints, capturing a photo at each waypoint. The photo is then transmitted to the GCS via the ESP32 WiFi module. The AoI of a PoI is calculated as the time elapsed since the GCS received this PoI's photo last time.

The AoI curve for the PoI with the greatest weighted AoI, under the TSP waypoints, is presented in Fig. 15(b). Notably, the MAX-AoI achieved by the LI2-generated waypoints is reduced by 28.440% compared to the TSP waypoints, by 23.482% compared to the SAC waypoints, and by a substantial 78.271% compared to the DDPG waypoints. Also, the results display a steady periodic pattern for AoI fluctuation, indicating the efficiency of LI2 in practice.

VII. CONCLUSION

This paper examines the widely-investigated problem of UAV route planning but uniquely focuses on a practical problem setting that has remained unaddressed thus far. The distinctive problem setting is from the specific demands of disaster response scenarios, which introduce two challenging constraints into the already complex UAV route planning problem. First, the flying environment may be replete with obstacles and thus can only be modeled as a general graph. Second, the UAV must be equipped with sufficient energy to complete the prescribed route. In response to this, we developed an innovative DRL-based solution called LI2. This solution iteratively enhances the current route with the node the DRL agent recommends and with various strategically designed operators. LI2 is versatile enough to accommodate the need of minimizing different AoI metrics.

We comprehensively evaluate LI2's performance using real-world datasets and field tests. The evaluation results demonstrate that LI2 significantly surpasses all existing DRL-based baselines and four well-known approximation algorithms. In the realm of future research, numerous intriguing avenues beckon exploration. One such avenue involves a comprehensive investigation into the potential of a UAV swarm in improving the information freshness within the PoI-monitoring problem. Additionally, delving into scenarios where the set of PoIs dynamically evolves over time adds a layer of complexity, catering to the ever-changing needs in post-disaster situations.

⁷The FM algorithm defines a semi-definite programming problem. We adopt the MOSEK optimizer [59] and do not set any limitation on the maximum number of iterations or the maximum running time.

⁸ $\mathcal{H}$ is the mixing time for the FM algorithm.

REFERENCES

- [1] A. Ferdowsi, M. A. Abd-Elmagid, W. Saad, and H. S. Dhillon, "Neural combinatorial deep reinforcement learning for age-optimal joint trajectory and scheduling design in UAV-assisted networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1250–1265, May 2021.
- [2] Z. Dai et al., "AoI-minimal UAV crowdsensing by model-based graph convolutional reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1029–1038.
- [3] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and Z. Gao, "UAV trajectory planning for AoI-minimal data collection in UAV-aided IoT networks by transformer," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1343–1358, Feb. 2023.
- [4] V. Tripathi, R. Talak, and E. Modiano, "Age optimal information gathering and dissemination on graphs," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 54–68, Jan. 2023.
- [5] S. Alamdari, E. Fata, and S. L. Smith, "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations," *Int. J. Robot. Res.*, vol. 33, no. 1, pp. 138–154, 2014.
- [6] K. Kalyanam, S. Manyam, A. Von Moll, D. Casbeer, and M. Pachter, "Scalable and exact MILP methods for UAV persistent visitation problem," in *Proc. IEEE Conf. Control Technol. Appl.*, 2018, pp. 337–342.
- [7] S. Hari, S. Rathinam, S. Darbha, K. Kalyanam, S. Manyam, and D. Casbeer, "Efficient computation of optimal UAV routes for persistent monitoring of targets," in *Proc. Int. Conf. Unmanned Aircr. Syst.*, 2019, pp. 605–614.
- [8] J.-M. Lien, S. Rodriguez, and M. Morales, "Persistent covering with latency and energy constraints," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 998–1003, Apr. 2021.
- [9] C. Zhou et al., "Deep RL-based trajectory planning for AoI minimization in UAV-assisted IoT," in *Proc. IEEE 11th Int. Conf. Wireless Commun. Signal Process.*, 2019, pp. 1–6.
- [10] M. Samir, C. Assi, S. Sharafeddine, D. Ebrahimi, and A. Ghrayeb, "Age of information aware trajectory planning of UAVs in intelligent transportation systems: A deep learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 12382–12395, Nov. 2020.
- [11] M. Wu, H. Chi, S. Gan, X. Wang, and C. Xu, "AoI optimal UAV trajectory planning: A deep recurrent reinforcement learning approach," in *Proc. IEEE 32nd Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2021, pp. 1–6.
- [12] M. Sun, X. Xu, X. Qin, and P. Zhang, "AoI-energy-aware UAV-assisted data collection for IoT networks: A deep reinforcement learning method," *IEEE Internet of Things J.*, vol. 8, no. 24, pp. 17275–17289, Dec. 2021.
- [13] E. Eldeeb, J. M. de Souza Sant'Ana, D. E. Pérez, M. Shehab, N. H. Mahmood, and H. Alves, "Multi-UAV path learning for age and power optimization in IoT with UAV battery recharge," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5356–5360, Apr. 2023.
- [14] Q. Chen, Z. Cai, L. Cheng, F. Wang, and H. Gao, "Joint near-optimal age-based data transmission and energy replenishment scheduling at wireless-powered network edge," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 770–779.
- [15] X. Chen, K. Gatsis, H. Hassani, and S. S. Bidokhti, "Age of information in random access channels," *IEEE Trans. Inf. Theory*, vol. 68, no. 10, pp. 6548–6568, Oct. 2022.
- [16] C. Li, S. Li, Y. Chen, Y. T. Hou, and W. Lou, "AoI scheduling with maximum thresholds," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 436–445.
- [17] O. S. Oubbati, M. Atiquzzaman, H. Lim, A. Rachedi, and A. Lakas, "Synchronizing UAV teams for timely data collection and energy transfer by deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 71, no. 6, pp. 6682–6697, Jun. 2022.
- [18] X. Fan, M. Liu, Y. Chen, S. Sun, Z. Li, and X. Guo, "RIS-assisted UAV for fresh data collection in 3D urban environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 632–647, Jan. 2023.
- [19] Z. Li, P. Tong, J. Liu, X. Wang, L. Xie, and H. Dai, "Learning-based data gathering for information freshness in UAV-assisted IoT networks," *IEEE Internet of Things J.*, vol. 10, no. 3, pp. 2557–2573, Feb. 2023.
- [20] G. Ahani, D. Yuan, and Y. Zhao, "Age-optimal UAV scheduling for data collection with battery recharging," *IEEE Commun. Lett.*, vol. 25, no. 4, pp. 1254–1258, Apr. 2021.
- [21] H. Hu, K. Xiong, G. Qu, Q. Ni, P. Fan, and K. B. Letaief, "AoI-minimal trajectory planning and data collection in UAV-assisted wireless powered IoT networks," *IEEE Internet of Things J.*, vol. 8, no. 2, pp. 1211–1223, Jan. 2021.
- [22] S. Zhang, H. Zhang, Z. Han, H. V. Poor, and L. Song, "Age of information in a cellular internet of UAVs: Sensing and communication trade-off design," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6578–6592, Oct. 2020.
- [23] Z.-H. Sun, X. Luo, E. Q. Wu, T.-Y. Zuo, Z.-R. Tang, and Z. Zhuang, "Monitoring scheduling of drones for emission control areas: An ant colony-based approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11699–11709, Aug. 2022.
- [24] E. S. Rigas, P. Kolios, M. Mavrovouniotis, and G. Ellinas, "Scheduling a fleet of drones for monitoring missions with spatial, temporal, and energy constraints," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15133–15145, Sep. 2022.
- [25] S. Jana and P. S. Mandal, "Approximation algorithms for drone delivery scheduling problem," in *Proc. Int. Conf. Netw. Syst.*, D. Mohaisen and T. Wies, Eds., Cham, Switzerland: Springer Nature, 2023, pp. 125–140.
- [26] E. Levner and V. Kats, "Maximizing the average environmental benefit of a fleet of drones under a periodic schedule of tasks," 2024.
- [27] X. Lin, Y. Yazicioglu, and D. Aksaray, "Robust planning for persistent surveillance with energy-constrained UAVs and mobile charging stations," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4157–4164, Apr. 2022.
- [28] A. Bogrybayeva, T. Yoon, H. Ko, S. Lim, H. Yun, and C. Kwon, "A deep reinforcement learning approach for solving the traveling salesman problem with drone," *Transp. Res. Part C: Emerg. Technol.*, vol. 148, 2023, Art. no. 103981. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0968090X22003941>
- [29] J. Luo, C. Li, Q. Fan, and Y. Liu, "A graph convolutional encoder and multi-head attention decoder network for TSP via reinforcement learning," *Eng. Appl. Artif. Intell.*, vol. 112, 2022, Art. no. 104848.
- [30] J. Perera, S.-H. Liu, M. Mernik, M. Črepinšek, and M. Ravber, "A graph pointer network-based multi-objective deep reinforcement learning algorithm for solving the traveling salesman problem," *Mathematics*, vol. 11, no. 2, p. 437, 2023.
- [31] Z. Ling, Y. Zhang, and X. Chen, "A deep reinforcement learning based real-time solution policy for the traveling salesman problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 5871–5882, Jun. 2023.
- [32] G. Fellek, A. Farid, S. Fujimura, O. Yoshie, and G. Gebreyesus, "GDGANet: Gated deep graph attention network with reinforcement learning for solving traveling salesman problem," *Neurocomputing*, vol. 579, 2024, Art. no. 127392.
- [33] L. Xin, W. Song, Z. Cao, and J. Zhang, "NeuroLKH: Combining deep learning model with Lin-Kernighan-Helsgaun heuristic for solving the traveling salesman problem," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 7472–7483.
- [34] J. Zheng, K. He, J. Zhou, Y. Jin, and C. M. Li, "Reinforced Lin-Kernighan-Helsgaun algorithms for the traveling salesman problems," *Knowl.-Based Syst.*, vol. 260, 2023, Art. no. 110144.
- [35] Y. Ruan, W. Cai, and J. Wang, "Combining reinforcement learning algorithm and genetic algorithm to solve the traveling salesman problem," *J. Eng.*, vol. 2024, no. 6, 2024, Art. no. e12393.
- [36] J. Zheng, J. Zhong, M. Chen, and K. He, "A reinforced hybrid genetic algorithm for the traveling salesman problem," *Comput. Operations Res.*, vol. 157, 2023, Art. no. 106249.
- [37] P. R. d O. Costa, J. Rhuggenaath, Y. Zhang, and A. Akcay, "Learning 2-OPT heuristics for the traveling salesman problem via deep reinforcement learning," in *Proc. Asian Conf. Mach. Learn.*, PMLR, 2020, pp. 465–480.
- [38] Y. Ma et al., "Learning to iteratively solve routing problems with dual-aspect collaborative transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 11096–11107.
- [39] H. Lu, X. Zhang, and S. Yang, "A learning-based iterative method for solving vehicle routing problems," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [40] F. Shan, J. Luo, R. Xiong, W. Wu, and J. Li, "Looking before crossing: An optimal algorithm to minimize UAV energy by speed scheduling with a practical flight energy model," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 1758–1767.
- [41] C. Rottondi, F. Malandrino, A. Bianco, C. F. Chiasserini, and I. Stavrakakis, "Scheduling of emergency tasks for multiservice UAVs in post-disaster scenarios," *Comput. Netw.*, vol. 184, 2021, Art. no. 107644.
- [42] Y. Wang et al., "Task offloading for post-disaster rescue in unmanned aerial vehicles networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1525–1539, Aug. 2022.
- [43] F. Malandrino, C.-F. Chiasserini, C. Casetti, L. Chiaraviglio, and A. Senacheribbe, "Planning UAV activities for efficient user coverage in disaster areas," *Ad Hoc Netw.*, vol. 89, pp. 177–185, 2019.
- [44] A. Amrallah, E. M. Mohamed, G. K. Tran, and K. Sakaguchi, "Optimization of UAV 3D trajectory in a post-disaster area using dual energy-aware bandits," *IEICE Commun. Exp.*, vol. 12, no. 8, pp. 403–408, 2023.

- [45] H. V. Abeywickrama, B. A. Jayawickrama, Y. He, and E. Dutkiewicz, "Comprehensive energy consumption model for unmanned aerial vehicles, based on empirical studies of battery performance," *IEEE Access*, vol. 6, pp. 58383–58394, 2018.
- [46] K. Helsgaun, "Solving the equality generalized traveling salesman problem using the Lin–Kernighan–Helsgaun algorithm," *Math. Program. Comput.*, vol. 7, pp. 269–287, 2015.
- [47] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.
- [48] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
- [49] H. Sak, A. Senior, and F. Beaufays, "Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition," 2014, *arXiv:1402.1128*.
- [50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [51] J. Weng et al., "Tianshou: A highly modularized deep reinforcement learning library," *J. Mach. Learn. Res.*, vol. 23, 2022, Art. no. 267. [Online]. Available: <http://jmlr.org/papers/v23/21--1127.html>
- [52] NASA, "Advanced rapid imaging and analysis (ARIA)," 2023. [Online]. Available: <https://aria.jpl.nasa.gov/>
- [53] V. Mnih et al., "Playing atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [54] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2016, pp. 1928–1937.
- [55] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 1861–1870.
- [56] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [57] A. Hakami, A. Kumar, S. J. Shim, and Y. A. Nahleh, "Application of soft systems methodology in solving disaster emergency logistics problems," *Int. J. Ind. Manuf. Eng.*, vol. 7, no. 12, pp. 2470–2477, 2013.
- [58] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!," 2018, *arXiv:1803.08475*.
- [59] M. ApS, "MOSEK optimizer API for Python. Version 10.0.46," 2023. [Online]. Available: <https://docs.mosek.com/latest/pythonapi/index.html>



Ziyao Huang received the BSc degree from Southeast University. He is currently working toward the PhD degree with the School of Computer Science and Engineering, Southeast University and the Department of Computer Science, City University of Hong Kong. His research interests include information freshness-aware scheduling problems, wireless communications, Unmanned Aerial Vehicle networks, and reinforcement learning.

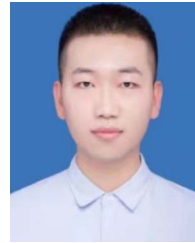


Weiwei Wu (Member, IEEE) received the BSc degree from the South China University of Technology, and the PhD degree from the Department of Computer Science, City University of Hong Kong (CityU), and University of Science and Technology of China (USTC), in 2011. He is currently a professor with the School of Computer Science and Engineering, Southeast University, China. He went to Nanyang Technological University (NTU, Mathematical Division, Singapore) for post-doctoral research, in 2012.

He has published more than 50 peer-reviewed papers in international conferences/journals, and serves as TPCs and reviewers for several top international journals and conferences. His research interests include optimizations and algorithm analysis, wireless communications, cloud computing, reinforcement learning, game theory, and network economics.



Kui Wu (Senior Member, IEEE) received the BSc and MSc degrees in computer science from Wuhan University, China, in 1990 and 1993, respectively, and the PhD degree in computing science from the University of Alberta, Canada, in 2002. In 2002, he joined the Department of Computer Science, University of Victoria, Canada, where he is currently a full professor. His research interests include network performance analysis, mobile and wireless networks, and network performance evaluation.



Hang Yuan is currently working toward the master's degree with the School of Computer Science and Engineering, Southeast University. His primary research interests encompass information freshness-aware scheduling problems, reinforcement learning, and Unmanned Aerial UAV networks.



Chenchen Fu (Member, IEEE) received the BSc degree from the South China University of Technology, and the PhD degree from the Department of Computer Science, City University of Hong Kong (CityU), and University of Science and Technology of China (USTC), in 2011. He is currently a professor with the School of Computer Science and Engineering, Southeast University, China. He went to Nanyang Technological University (NTU, Mathematical Division, Singapore) for post-doctoral research, in 2012.

He has published more than 50 peer-reviewed papers in international conferences/journals, and serves as TPCs and reviewers for several top international journals and conferences. His research interests include optimizations and algorithm analysis, wireless communications, cloud computing, reinforcement learning, game theory, and network economics.



Feng Shan received the PhD degree in computer science from Southeast University, Nanjing, China, in 2015. He was a visiting student with the School of Computing and Engineering, University of Missouri-Kansas City, Kansas City, Missouri, from 2010 to 2012. He is currently an associate professor with the School of Computer Science and Engineering, Southeast University. His research interests include the areas of Internet of Things, wireless networks, swarm intelligence, and algorithm design and analysis.



Jianping Wang (Fellow, IEEE) received the BS and MS degrees in computer science from Nankai University, Tianjin, China, in 1996 and 1999, respectively, and the PhD degree in computer science from the University of Texas at Dallas, Richardson, Texas, in 2003. She is currently a professor with the Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong. Her research interests include dependable networking, optical networks, cloud computing, service oriented networking, and data center networks.



Junzhou Luo (Member, IEEE) received the BS degree in applied mathematics, and the MS and PhD degrees in computer network, all from Southeast University, China, in 1982, 1992, and 2000, respectively. He is a full professor with the School of Computer Science and Engineering, Southeast University, Nanjing, China. He is a member of the IEEE Computer Society and co-chair of IEEE SMC Technical Committee on computer supported cooperative work in design, and he is a member of the ACM and chair of ACM SIGCOMM China. His research interests are

next generation network architecture, network security, cloud computing, and wireless LAN.